A Circuit-Based Tutorial on Shor's Algorithm for Finance Students

Alejandro Reynoso

June 15, 2026

Abstract

Shor's algorithm is one of the landmark results in quantum computing: a sufficiently large fault-tolerant quantum computer could factor large integers in polynomial time [1]. This matters in finance because public-key cryptography supports authentication, digital signatures, secure messaging, payment infrastructure, digital-asset custody, and long-lived confidential records.

This tutorial is written for Master of Finance students who already know the basic quantum ideas of superposition and interference. The goal is not to prove Shor's algorithm in full mathematical detail. Instead, the goal is to make the circuit logic transparent: why factoring becomes period finding, how modular exponentiation is placed inside a reversible circuit, how the inverse quantum Fourier transform amplifies period information, and how classical post-processing converts a measurement into factors.

1 Absolute Beginner's Explanation

Imagine that a large number is a locked box, and its prime factors are the two keys that explain how the box was made. If the number is small, such as 15, opening the box is easy because we can quickly see that 15 equals 3 times 5. If the number has hundreds or thousands of digits, trying possible factors one by one becomes impractical. Modern public key cryptography uses this imbalance. It is easy to multiply two large primes, but difficult to reverse the product and recover the primes.

Shor's algorithm attacks this problem indirectly. It does not ask a quantum computer to try factor after factor. Instead, it turns factoring into the search for a repeating pattern. Pick a number a that shares no factor with the number N we want to factor. Then look at the sequence a to the power x, reduced modulo N. Reduced modulo N simply means keeping the remainder after division by N. Because there are only finitely many possible remainders, the sequence must eventually repeat. The length of the repeating cycle is called the period.

The surprising part is that knowing this period can reveal the factors of N. Once the period is found, ordinary classical arithmetic, especially greatest common divisors, can often split N into its prime components. Therefore, the quantum computer's main job is not factoring directly. Its job is period finding.

The circuit has two registers. The counting register stores many possible exponents x. The function register stores the value of a to the power x modulo N. Hadamard gates put the counting register into superposition, so the circuit represents many possible x values at once. Modular exponentiation then computes the corresponding remainders coherently, without measuring and destroying the superposition.

At this point the period is hidden in the spacing of the amplitudes. Values separated by the period lead to the same remainder. The inverse quantum Fourier transform is the circuit component

that converts this spacing into peaks in the measurement probabilities. This is where interference does the useful work: wrong possibilities tend to cancel, while values related to the true period reinforce one another.

Finally, measuring the counting register gives a number m. The ratio m divided by the size of the register is usually close to a fraction s over r, where r is the unknown period. A classical continued fraction calculation extracts a candidate r. The algorithm checks the candidate. If it works, greatest common divisors produce the factors. If it fails, the circuit can be run again. This makes Shor's algorithm a hybrid procedure: quantum mechanics finds the period clue, and classical number theory verifies and converts it into factors. A useful mental model is frequency extraction. You are not asking the machine to print every value of the modular sequence. You are asking it to make the cycle length visible. The quantum circuit prepares the signal, the inverse QFT analyzes its frequency content, and classical arithmetic checks the answer carefully before accepting any result.

2 What Problem Are We Solving?

The input to Shor's algorithm is a composite integer N. The objective is to find nontrivial factors of N [2]. For example,

$$N = 15 = 3 \times 5.$$

Small examples are easy by inspection. The importance of the problem comes from large integers. In RSA, for example, the public modulus has the form

$$N = pq,$$

where p and q are large primes. Multiplying p and q is easy, but recovering p and q from N is believed to be hard for classical computers when the numbers are large enough.

Finance intuition. Factoring is a one-way function in the same broad sense as many financial operations: it is easy to combine the inputs, but hard to reverse the operation without special information. RSA relies on this asymmetry. Shor's algorithm is important because it changes the computational status of that reverse operation on a sufficiently powerful quantum computer.

Shor's key insight is that the quantum computer does not search directly for the prime factors. Instead, it solves a structured periodicity problem. The entire algorithm can be summarized as

$$\boxed{\text{Factoring} \longrightarrow \text{period finding} \longrightarrow \text{greatest common divisors.}}$$

The quantum circuit is responsible for the middle step: finding a period.

3 The Reduction from Factoring to Period Finding

Choose an integer a satisfying

$$1 < a < N$$

and compute

$$\gcd(a, N).$$

If $\gcd(a, N) > 1$, then a factor has already been found and the quantum circuit is not needed. The interesting case is

$$\gcd(a, N) = 1.$$

Now define the modular function

$$f(x) = a^x \bmod N.$$

Because the values of $f(x)$ live in a finite modular arithmetic system, the sequence eventually repeats. The smallest positive integer r such that

$$a^r \equiv 1 \pmod{N}$$

is called the **order** or **period** of a modulo N.

If the period r is even, then

$$a^r - 1 = \left(a^{r/2} - 1\right)\left(a^{r/2} + 1\right).$$

Since $a^r \equiv 1 \pmod{N}$, the number N divides $a^r - 1$. Under the usual nondegenerate condition

$$a^{r/2} \not\equiv -1 \pmod{N},$$

the two greatest common divisors

$$\gcd\left(a^{r/2} - 1, N\right) \quad \text{and} \quad \gcd\left(a^{r/2} + 1, N\right)$$

often reveal nontrivial factors of N.

Key idea. The quantum part of Shor's algorithm finds the period r. The final conversion from r to factors uses ordinary classical arithmetic.

4 A Fully Worked Warm-Up: Factoring 15

Let

$$N = 15, \qquad a = 2.$$

The choice is valid because

$$\gcd(2, 15) = 1.$$

Now compute successive powers of 2 modulo 15:

x	2^x	$2^x \bmod 15$
0	1	1
1	2	2
2	4	4
3	8	8
4	16	1
5	32	2
6	64	4
7	128	8

The modular sequence is

$$1, 2, 4, 8, 1, 2, 4, 8, \ldots$$

so the period is

$$r = 4.$$

Because r is even,

$$2^{r/2} = 2^2 = 4.$$

The factors are then obtained from

$$\gcd(4-1, 15) = \gcd(3, 15) = 3$$

and

$$\gcd(4+1, 15) = \gcd(5, 15) = 5.$$

Therefore,

$$15 = 3 \times 5.$$

This example shows the entire classical logic. The only missing piece is how a quantum circuit finds r efficiently when N is large.

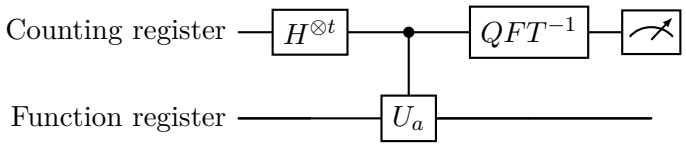
5 The Quantum Task in One Sentence

The quantum circuit prepares many possible exponents x in superposition, computes $a^x \bmod N$ coherently, and uses interference to make the period visible in the measurement probabilities.

The high-level architecture is

$$\boxed{\text{Hadamards} \to \text{controlled modular exponentiation} \to QFT^{-1} \to \text{measurement.}}$$

In circuit form:



This compact picture hides many implementation details, but it captures the logic of the algorithm.

6 The Two Quantum Registers

Shor's period-finding circuit uses two registers.

6.1 Counting Register

The first register stores possible exponents x. It is also called the control register or phase register. If it has t qubits, it can represent

$$2^t$$

possible values:

$$0, 1, 2, \ldots, 2^t - 1.$$

It begins in

$$|0\rangle^{\otimes t}.$$

6.2 Function Register

The second register stores the modular function value

$$a^x \bmod N.$$

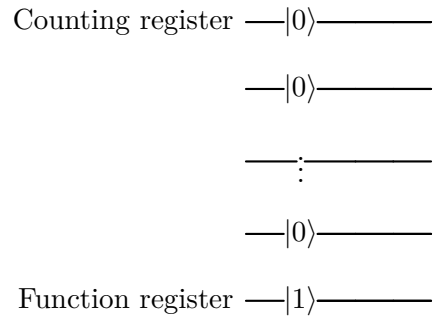
It starts in the computational basis state

$$|1\rangle .$$

The initial joint state is therefore

$$|0\rangle^{\otimes t} |1\rangle .$$

A schematic view is



Key idea. The counting register asks "which exponent x?" The function register stores "what is a^x mod N?"

7 Step 1: Create a Uniform Superposition

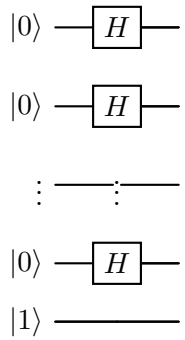
Apply a Hadamard gate to every qubit in the counting register. For one qubit,

$$H |0\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}.$$

For t qubits,

$$H^{\otimes t} |0\rangle^{\otimes t} = \frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle .$$

The circuit begins as



The state after the Hadamard gates is

$$\frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle |1\rangle .$$

The circuit now represents all possible exponents x at the same time. However, this is not a free lunch: measuring immediately would return only one random value of x. The purpose of the later circuit blocks is to arrange the amplitudes so that measurement is no longer random noise, but is biased toward information about the period.

8 Step 2: Compute Modular Exponentiation Coherently

The next block computes

$$a^x \bmod N$$

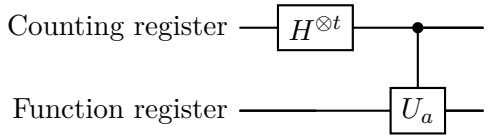
without measuring x. The desired transformation is

$$|x\rangle\,|1\rangle \longrightarrow |x\rangle\,|a^x \bmod N\rangle\,.$$

After this block, the joint state is

$$\frac{1}{\sqrt{2^t}} \sum_{x=0}^{2^t-1} |x\rangle\,|a^x \bmod N\rangle\,.$$

At a high level, modular exponentiation is represented by a controlled unitary operation:



Here U_a is modular multiplication:

$$U_a\,|y\rangle = |ay \bmod N\rangle\,.$$

The controlled operation is built from powers

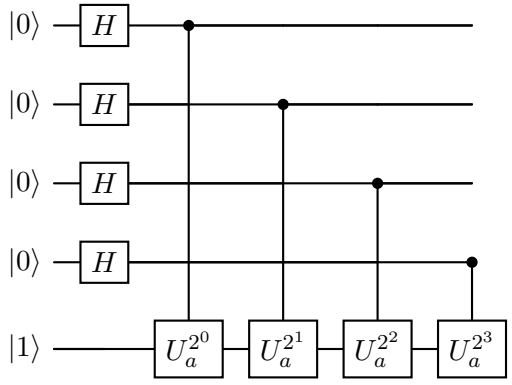
$$U_a^{2^0}, \quad U_a^{2^1}, \quad U_a^{2^2}, \quad \ldots$$

because every exponent x has a binary expansion

$$x = x_0 2^0 + x_1 2^1 + x_2 2^2 + \cdots\,.$$

If the bit x_j equals 1, the circuit applies the block $U_a^{2^j}$. If $x_j = 0$, it does not.

A four-counting-qubit schematic is



Key idea. Superposition supplies many exponent values. Controlled modular exponentiation writes the periodic function values next to those exponent values without collapsing the superposition.

9 Why Modular Exponentiation Is Reversible

Quantum gates must be reversible. The map

$$|y\rangle \longrightarrow |ay \bmod N\rangle$$

is reversible when

$$\gcd(a, N) = 1.$$

In that case, multiplication by a modulo N is a permutation of the allowed basis states. Every output has a unique input because a has a modular inverse.

This explains why the algorithm begins by checking $\gcd(a, N)$. If the greatest common divisor is not 1, a factor has already been found. If it is 1, the modular multiplication operation can be embedded in a reversible quantum circuit.

10 Where the Period Is Hidden

For the example $N = 15$, $a = 2$, and $t = 4$, the counting register has

$$2^t = 16$$

possible values. After modular exponentiation, the state has the pattern

$$\begin{aligned}\frac{1}{4}(&|0\rangle|1\rangle + |1\rangle|2\rangle + |2\rangle|4\rangle + |3\rangle|8\rangle \\ &+ |4\rangle|1\rangle + |5\rangle|2\rangle + |6\rangle|4\rangle + |7\rangle|8\rangle \\ &+ |8\rangle|1\rangle + |9\rangle|2\rangle + |10\rangle|4\rangle + |11\rangle|8\rangle \\ &+ |12\rangle|1\rangle + |13\rangle|2\rangle + |14\rangle|4\rangle + |15\rangle|8\rangle).\end{aligned}$$

The function values repeat every four exponents. For example, the function-register value $|1\rangle$ is paired with

$$|0\rangle, \quad |4\rangle, \quad |8\rangle, \quad |12\rangle$$

in the counting register. These exponent values are separated by the period $r = 4$.

If one conceptually measured the function register and obtained $|1\rangle$, the counting register would collapse to a superposition proportional to

$$|0\rangle + |4\rangle + |8\rangle + |12\rangle.$$

The same periodic structure is present even when the function register is not measured. The inverse QFT acts on the counting register and converts this hidden spacing into peaks in the measurement distribution.

Finance intuition. Think of the modular function values as a repeating signal. The quantum circuit does not list the whole signal for us. Instead, it prepares a state whose amplitudes contain the spacing pattern, then uses a Fourier-type transformation to make the frequency information measurable.

11 Step 3: Use the Inverse QFT to Turn Spacing into Peaks

The inverse quantum Fourier transform, written

$$QFT^{-1},$$

acts on the counting register. Its role is to transform periodic structure in the amplitudes into large measurement probabilities near numbers m satisfying [3]

$$\frac{m}{2^t} \approx \frac{s}{r},$$

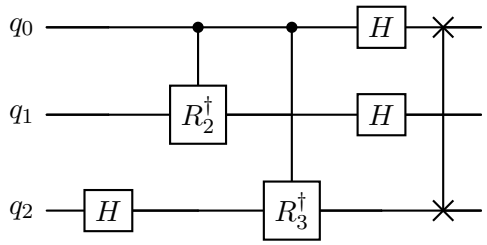
where

$$s \in \{0, 1, \ldots, r-1\}.$$

The symbols mean:

- m is the integer measured from the counting register;
- 2^t is the number of computational basis states in that register;
- r is the unknown period;
- s labels one of the Fourier peaks associated with the period.

For three qubits, a simplified inverse QFT circuit contains Hadamard gates, controlled inverse phase rotations, and a final reversal of qubit order:



The phase-rotation gates are

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}, \qquad R_k^\dagger = \begin{pmatrix} 1 & 0 \\ 0 & e^{-2\pi i/2^k} \end{pmatrix}.$$

The inverse QFT does not usually output r directly. It outputs a binary approximation to a fraction s/r. Classical post-processing then reconstructs a candidate denominator.

12 Step 4: Measure the Counting Register

Return to the small example with

$$N = 15, \qquad a = 2, \qquad r = 4, \qquad t = 4.$$

Then

$$2^t = 16.$$

The inverse QFT creates high probabilities near

$$\frac{s}{r} \in \left\{0, \frac{1}{4}, \frac{2}{4}, \frac{3}{4}\right\}.$$

Multiplying these fractions by 16 gives the likely measurement outcomes

$$m \in \{0, 4, 8, 12\}.$$

The table below shows the interpretation:

m	binary output	$m/16$	information obtained
0	0000	0	usually not useful
4	0100	1/4	suggests $r = 4$
8	1000	1/2	may suggest only a divisor of r
12	1100	3/4	suggests $r = 4$

If the measurement gives $m = 4$, then

$$\frac{m}{2^t} = \frac{4}{16} = \frac{1}{4}.$$

The denominator suggests

$$r = 4.$$

If the measurement gives $m = 8$, then

$$\frac{m}{2^t} = \frac{8}{16} = \frac{1}{2}.$$

This may suggest $r = 2$, but it is only a divisor of the true period. The classical check rejects it because

$$2^2 \equiv 4 \not\equiv 1 \pmod{15}.$$

In that case, the algorithm repeats the quantum circuit or chooses a new value of a.

Key idea. A measurement gives a clue about r, not a guaranteed final answer. Shor's algorithm is probabilistic, so repeated runs are part of the design.

13 Step 5: Recover the Period with Continued Fractions

In realistic cases, 2^t is not usually a perfect multiple of r. The measured ratio

$$\frac{m}{2^t}$$

is therefore only an approximation to

$$\frac{s}{r}.$$

A classical continued-fraction algorithm finds a simple rational approximation. For example, if

$$\frac{m}{2^t} \approx 0.333333,$$

then a natural rational approximation is

$$\frac{1}{3}.$$

The denominator 3 becomes a candidate period.

The candidate is never accepted blindly. It must pass the modular check

$$a^r \equiv 1 \pmod{N}.$$

If it fails, the algorithm tries again.

14 The Complete Hybrid Algorithm

Shor's algorithm is a hybrid quantum-classical algorithm. The quantum computer performs period finding; the classical computer handles selection, greatest common divisors, continued fractions, and verification.

To factor an odd composite integer N:

1. Choose a random integer a with $1 < a < N$.
2. Compute $d = \gcd(a, N)$.
3. If $d > 1$, stop: d is a nontrivial factor.
4. If $d = 1$, run the quantum period-finding circuit for $f(x) = a^x \bmod N$.
5. Convert the measured value m into a candidate period r using continued fractions.
6. Check whether r is even and whether $a^r \equiv 1 \pmod{N}$.
7. Check that $a^{r/2} \not\equiv -1 \pmod{N}$.
8. Compute
$$p = \gcd\left(a^{r/2} - 1, N\right), \qquad q = \gcd\left(a^{r/2} + 1, N\right).$$
9. If p and q are nontrivial factors, stop. Otherwise, choose another a or repeat the quantum run.

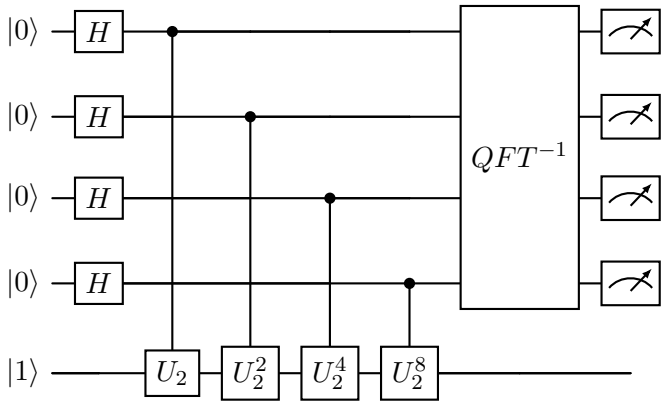
The workflow is

$$\boxed{\text{choose } a \to \text{quantum period finding} \to \text{continued fractions} \to \text{GCDs} \to \text{factors.}}$$

15 A Circuit-Level View of the Example $N = 15$

For
$$N = 15, \qquad a = 2,$$
the modular multiplication operator is
$$U_2 \left|y\right\rangle = \left|2y \bmod 15\right\rangle.$$
With four counting qubits, the controlled powers are
$$U_2, \qquad U_2^2, \qquad U_2^4, \qquad U_2^8.$$
These correspond to multiplication by
$$2^1 \bmod 15 = 2, \qquad 2^2 \bmod 15 = 4, \qquad 2^4 \bmod 15 = 1, \qquad 2^8 \bmod 15 = 1.$$

A conceptual four-counting-qubit circuit is



This diagram deliberately treats each $U_2^{2^j}$ as a single block. In a physical quantum processor, each block must be decomposed into elementary one-qubit gates, two-qubit gates, and reversible arithmetic subroutines.

16 What Is Hidden Inside the Modular Exponentiation Block?

The modular exponentiation block is usually the most expensive part of Shor's algorithm. It may contain reversible circuits for:

- addition;
- controlled addition;
- modular reduction;
- modular multiplication;
- ancillary-qubit management;
- uncomputation of temporary values.

The last item is especially important. A quantum circuit may use temporary ancillary qubits to store intermediate calculations. Before the circuit ends, those temporary values should normally be erased reversibly by applying the inverse of the calculation that created them. This process is called **uncomputation**.

Uncomputation prevents unwanted information from remaining entangled with the principal registers. Without it, the interference pattern in the counting register can be damaged.

Key idea. The elegant high-level circuit hides a major engineering challenge: modular arithmetic must be implemented reversibly, coherently, and with enough error correction to preserve interference.

17 Phase Estimation Interpretation

The same circuit can also be described as quantum phase estimation. This viewpoint is useful because it explains why the inverse QFT appears [4].

Consider the modular multiplication operator

$$U_a |y\rangle = |ay \bmod N\rangle .$$

For the relevant subspace, this operator has eigenstates $|u_s\rangle$ satisfying

$$U_a |u_s\rangle = e^{2\pi i s/r} |u_s\rangle .$$

The corresponding phase is

$$\phi_s = \frac{s}{r}.$$

Quantum phase estimation estimates ϕ_s. The inverse QFT converts phase information into a binary measurement from which a rational approximation to s/r can be recovered.

This is why the output is not usually "the period is r" directly. The output is closer to "the phase is approximately s/r," and the classical continued-fraction step extracts the denominator.

18 Why Several Runs May Be Required

Shor's algorithm is probabilistic. A single run can fail for several reasons:

- the measurement corresponds to $s = 0$, which gives no useful denominator;
- s and r have a common factor, so the denominator is only a divisor of r;
- the continued-fraction step proposes a candidate that fails $a^r \equiv 1 \pmod{N}$;
- the chosen a produces an odd period;
- $a^{r/2} \equiv -1 \pmod{N}$, which prevents the GCD step from producing nontrivial factors;
- hardware noise corrupts the circuit output.

This does not undermine the algorithm. Many quantum algorithms are probabilistic: they are designed to be repeated until a verifiable answer is obtained.

19 Quantum Resources and Practical Limits

For an n-bit integer, Shor's algorithm uses a number of logical qubits and gates that grows polynomially with n. That polynomial scaling is the theoretical breakthrough.

However, practical implementations require:

1. enough logical qubits for the counting and function registers;
2. additional ancillary qubits for reversible arithmetic;
3. many controlled modular multiplication gates;
4. deep circuits with coherent phase relationships;
5. quantum error correction for large cryptographic instances.

Factoring large RSA moduli therefore requires fault-tolerant quantum computers, not merely small noisy devices. The finance implication is strategic rather than immediate panic: institutions should understand which systems depend on factorization or discrete logarithms and plan migration paths toward post-quantum cryptography for long-lived sensitive data [5].

20 A Finance-Oriented Analogy

A useful analogy is frequency extraction from a periodic signal. Suppose a market-related time series repeats every r periods, but r is unknown. A classical analyst might sample the sequence and compare many possible cycle lengths. A Fourier transform helps reveal the dominant frequency.

Shor's circuit has a similar high-level structure:

1. the counting register represents many time indices x in superposition;
2. modular exponentiation evaluates a periodic function at those indices;
3. the repeated structure is encoded into quantum amplitudes and phases;
4. the inverse QFT concentrates probability near frequencies associated with the period;
5. measurement and classical processing infer the period.

The analogy is useful but limited. Shor's algorithm is not a forecasting method, and its periodic function is not a financial time series. The point of the analogy is only to make the circuit logic intuitive: periodic structure is converted into frequency information.

21 Summary of the Circuit

The central circuit architecture is

$$\boxed{\text{Hadamards} \to \text{controlled modular exponentiation} \to QFT^{-1} \to \text{measurement.}}$$

Each component has a specific role:

Component	**Purpose**
Hadamard gates	Prepare a uniform superposition of possible exponents x.
Controlled modular exponentiation	Compute $a^x \bmod N$ coherently and encode periodic structure.
Inverse QFT	Convert the hidden period into measurement peaks near s/r.
Measurement	Produce an integer m whose ratio $m/2^t$ approximates s/r.
Continued fractions	Recover a candidate denominator r.
Greatest common divisors	Convert a valid even period into factors of N.

22 Conclusion

Shor's algorithm is powerful because it reframes a difficult arithmetic problem as a problem about periodic structure. Factoring a large integer looks, at first, like a search through a huge space of possible divisors. Shor's insight is that one can instead choose a number a, study the repeating behavior of a to the power x modulo N, and then use the period of that repetition to expose nontrivial factors of N. The hard part is therefore moved from direct factor search to period finding.

The quantum circuit is designed exactly for that task. Hadamard gates create a superposition over many exponent values. Controlled modular exponentiation evaluates the periodic function

while preserving coherence. The inverse quantum Fourier transform then converts the hidden repetition into a measurement distribution with peaks near fractions of the form s over r. Measurement gives a classical integer, and continued fractions turn that integer into a candidate period. The candidate is checked, and greatest common divisors convert a successful period into factors.

For finance students, the most important lesson is not that a quantum computer magically tries every factor at once. That picture is misleading. The advantage comes from structure and interference. Superposition provides a broad set of possible inputs, but interference determines which outputs become likely. The circuit is useful because the modular function has a rigid repeating pattern, and the inverse QFT is tailored to reveal that pattern. Without the period structure, superposition alone would not solve the problem.

The security implication is also specific. Shor's algorithm threatens cryptographic systems whose security depends on factoring or discrete logarithms, including RSA and elliptic curve cryptography. It does not break every cryptographic primitive, and it does not imply that today's noisy quantum devices can immediately compromise modern financial infrastructure. Large scale attacks would require fault tolerant quantum computers with enough logical qubits, deep reversible arithmetic circuits, and strong error correction. Nevertheless, financial institutions care because encrypted records, signatures, identity systems, and settlement infrastructure often need to remain secure for many years.

This is why the practical response is preparation rather than panic. Risk teams should identify where vulnerable public key systems are used, estimate how long protected data must remain confidential, and follow the transition toward post quantum cryptography. From a technical viewpoint, Shor's algorithm is a beautiful example of hybrid computation: quantum mechanics extracts period information, while classical computation performs verification and factor recovery. From a finance viewpoint, it is a reminder that computational assumptions are part of market infrastructure. When those assumptions change, models of operational, cyber, and systemic risk must change as well. Understanding the circuit logic helps professionals see both the promise and the limits of quantum computing clearly. The central message is therefore balanced. Shor's algorithm is mathematically disruptive, but operationally demanding. It is neither science fiction nor an overnight emergency. It is a concrete algorithm whose circuit structure can be understood step by step, and whose strategic consequences should be managed with disciplined planning across technology, risk, compliance, and governance teams.

References

[1] Peter W. Shor, "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.

[2] Michael A. Nielsen and Isaac L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press, 2010.

[3] N. David Mermin, *Quantum Computer Science: An Introduction*, Cambridge University Press, 2007.

[4] Alexei Yu. Kitaev, Alexander H. Shen, and Mikhail N. Vyalyi, *Classical and Quantum Computation*, Graduate Studies in Mathematics, vol. 47, American Mathematical Society, 2002.

[5] National Institute of Standards and Technology, *Post-Quantum Cryptography Standards: FIPS 203, FIPS 204, and FIPS 205*, 2024.